

Inteligência Artificial | FGA0021 | Turma 01 | 2026.1

Professor: Dr. Fabiano Araujo Soares

Aluno: Bruno Cruz Garcia Rosa

Matrícula: 222006605

Banco de Conhecimento

1. Introdução

Neste projeto, foi desenvolvido um sistema especialista simples para diagnóstico de problemas em computadores. Utilizando os conceitos de **banco de conhecimentos**, o sistema realiza perguntas ao usuário sobre sintomas apresentados pelo computador, armazenando as respostas como fatos dentro de uma base de conhecimento. A partir desses fatos, **regras de inferência** são aplicadas para identificar possíveis problemas de hardware, software, temperatura, armazenamento, memória, rede e outros componentes do sistema.

O projeto foi desenvolvido utilizando a linguagem Python juntamente com a biblioteca Pygame para construção da interface gráfica. A interface permite acompanhar visualmente:

- Perguntas realizadas;
- Fatos armazenados;
- Diagnósticos encontrados;

Além disso, foi utilizada a biblioteca **Pygame** para a construção da interface gráfica e visualização da exploração dos algoritmos no labirinto, permitindo acompanhar visualmente a expansão dos nós e o caminho encontrado. Para execução correta do projeto, é necessária a instalação prévia da biblioteca Pygame através do comando:

- `pip install pygame`
- ou:
- `pip install pygame-ce`

2. Definição do Problema

O problema abordado neste projeto consiste na identificação automática de possíveis falhas em computadores através da utilização de um sistema baseado em conhecimento. O sistema recebe informações fornecidas pelo usuário sobre sintomas observados no computador, como:

- ausência de vídeo;

- travamentos;
- superaquecimento;
- falhas de armazenamento.

Com base nessas informações, o sistema utiliza regras previamente definidas para inferir possíveis causas do problema apresentado. O objetivo principal é simular o funcionamento básico de um sistema especialista, utilizando:

- Base de conhecimento;
- Representação de fatos;
- Regras condicionais;
- Inferência lógica.

O projeto não realiza diagnóstico físico real do computador, mas demonstra como conceitos de banco de conhecimentos podem ser utilizados para automatizar processos de tomada de decisão baseados em conhecimento especializado.

2.1. Formalização do Problema:

- **Ações:** As ações disponíveis consistem nas respostas fornecidas pelo usuário para cada pergunta realizada pelo sistema. Cada resposta adiciona novos fatos à base de conhecimento, podendo ser positivos ou negativos.
- **Objetivo:** O objetivo consiste em identificar automaticamente possíveis problemas no computador através da aplicação das regras de inferência presentes na base de conhecimento.
- **Base de Conhecimento:** A base de conhecimento é composta por:
 - Fatos;
 - Regras condicionais;
 - Diagnósticos possíveis;
 - Níveis de confiança.
- **Inferência:** A inferência ocorre quando os fatos fornecidos pelo usuário satisfazem as condições de uma regra, permitindo gerar um diagnóstico correspondente.

2.2. Complexidade do Ambiente

O ambiente do problema possui baixa complexidade estrutural, porém apresenta grande quantidade de combinações possíveis entre sintomas e diagnósticos. Cada resposta fornecida pelo usuário altera o conjunto de fatos conhecidos pelo sistema, permitindo diferentes ativações de regras e diferentes diagnósticos possíveis.

Além disso, múltiplas regras podem ser satisfeitas simultaneamente, fazendo com que o sistema apresente mais de um diagnóstico com diferentes níveis de confiança. O crescimento do número de regras aumenta diretamente a quantidade de inferências possíveis, tornando o sistema mais completo, porém aumentando a complexidade da manutenção da base de conhecimento.

3. Algoritmos Desenvolvidos

3.1. Base de Conhecimento

A base de conhecimento do sistema foi desenvolvida utilizando estruturas de dados em Python para representar regras, perguntas, diagnósticos e fatos conhecidos. As regras foram implementadas através de listas de dicionários, onde cada regra possui:

- condições necessárias (*if*);
- diagnóstico resultante (*then*);
- nível de confiança (*confidence*).

Por Exemplo, caso o computador ligue, não apresente vídeo e não emita sinais sonoros (beep), o sistema infere um possível problema relacionado à placa de vídeo. Além disso, o sistema utiliza uma estrutura de mapeamento (*fact_labels*) para converter fatos internos em textos amigáveis apresentados na interface gráfica.

3.2. Inferência

O mecanismo de inferência foi desenvolvido utilizando encadeamento progressivo (Forward Chaining). Nesse modelo, os fatos fornecidos pelo usuário são adicionados à base de fatos e comparados continuamente com as regras da base de conhecimento. O funcionamento ocorre da seguinte forma:

1. O sistema realiza perguntas ao usuário;
2. Cada resposta gera um fato positivo ou negativo;
3. Os fatos são armazenados;
4. O sistema percorre todas as regras;
5. Caso todas as condições de uma regra sejam satisfeitas, um diagnóstico é gerado.

A verificação das regras ocorre através da comparação entre os fatos atuais e as condições presentes em cada regra. Quando a condição é satisfeita, o diagnóstico correspondente é adicionado à lista de diagnósticos encontrados juntamente com seu percentual de confiança. O sistema também permite múltiplos diagnósticos simultaneamente, possibilitando identificar diferentes problemas relacionados ao mesmo conjunto de sintomas.

Os diagnósticos são posteriormente ordenados de acordo com o maior nível de confiança, permitindo destacar visualmente o problema mais provável. Caso nenhuma regra seja satisfeita, o sistema apresenta um diagnóstico inconclusivo.

3.3. Pygame

A biblioteca Pygame foi utilizada para construção da interface gráfica e visualização da execução do algoritmo. A interface permite:

- responder perguntas através de botões;
- visualizar os fatos conhecidos;
- acompanhar os diagnósticos inferidos;
- visualizar níveis de confiança;
- reiniciar a execução do sistema.

Os fatos positivos e negativos são apresentados visualmente com cores diferentes. Os diagnósticos são exibidos em uma área separada da interface, sendo destacado em verde o diagnóstico com maior nível de confiança.

3.4. Execução Principal (Main)

O arquivo principal do projeto é responsável por controlar todo o fluxo de execução do sistema especialista. Inicialmente, são carregadas: As regras da base de conhecimento, perguntas, diagnósticos possíveis, textos exibidos na interface.

Durante a execução o sistema faz perguntas ao usuário, que responde clicando nos botões de “Sim” ou “Não”. Ao final, os diagnósticos são mostrados na tela. Esse arquivo também controla alguns eventos usados no Pygame.

Figura 1 - Programa em execução mostrando os diagnósticos e fatos obtidos.

